

CS6886 — Assignment 2

MobileNet-v2 on CIFAR-10: Training and Custom Quantization

Adhil Ahmed (da24s023) • September 8, 2026

GitHub: https://github.com/starkAhmed43/CS6886_A2

1 Q1 — Training Baseline

(a) Data preparation and transforms

We use CIFAR-10 (50,000 train, 10,000 test, 32×32 RGB, 10 classes). We normalize with the per-channel CIFAR-10 statistics and augment only the training split.

1. **Train:** `RandomCrop(32, padding=4), RandomHorizontalFlip(), ToTensor(), Normalize(mean, std)`.
2. **Test:** `ToTensor(), Normalize(mean, std)` only (no augmentation).
3. **Statistics:** `mean = (0.4914, 0.4822, 0.4465)`, `std = (0.2470, 0.2435, 0.2616)`.

`RandomCrop` with 4-pixel reflection padding teaches translation invariance. `RandomHorizontalFlip` adds mirror symmetry. `ToTensor` scales pixels to $[0, 1]$. `Normalize` centers each channel near zero, which stabilizes training.

(b) Model and training strategy

The stock network downsamples five times (tuned for 224×224 ImageNet), collapsing a 32×32 input to a 1×1 map. We relax the earliest three stride-2 convolutions to stride 1 (`stride_relax=3`), keeping an 8×8 final feature map. This adds no parameters and only changes *where* downsampling happens. Further details about my MobileNet-v2 configuration can be found in Table 1.

Setting	Value
Architecture	MobileNet-v2 (torchvision, from scratch, <code>weights=None</code>)
Width multiplier / dropout	1.0 / 0.2
CIFAR adaptation	<code>stride_relax=3</code> (8×8 final feature map)
Norm / activation	default <code>BatchNorm2d</code> / <code>ReLU6</code>
Parameters	~ 2.2 M
Optimizer	SGD, <code>lr=0.1</code> , <code>momentum=0.9</code> , Nesterov, <code>weight_decay=5e-4</code>
LR schedule	5-epoch linear warmup, then cosine annealing (200 epochs)
Batch size / label smoothing	128 / 0.1
Precision / seed	<code>bf16-mixed</code> / 42

Table 1: MobileNet-v2 configuration and training strategy.

(c) Results and failure modes

Final test top-1 accuracy: 93.98% (`stride_relax=3`). Table 2 shows the effect of the CIFAR downsampling adaptation. The loss and accuracy curves appear in Figure 1, and the confusion matrix in Figure 2.

<code>stride_relax</code>	final map	test top-1
1	2×2	90.65%
2	4×4	92.47%
3	8×8	93.98%

Table 2: A larger final feature map keeps more spatial detail and raises accuracy.

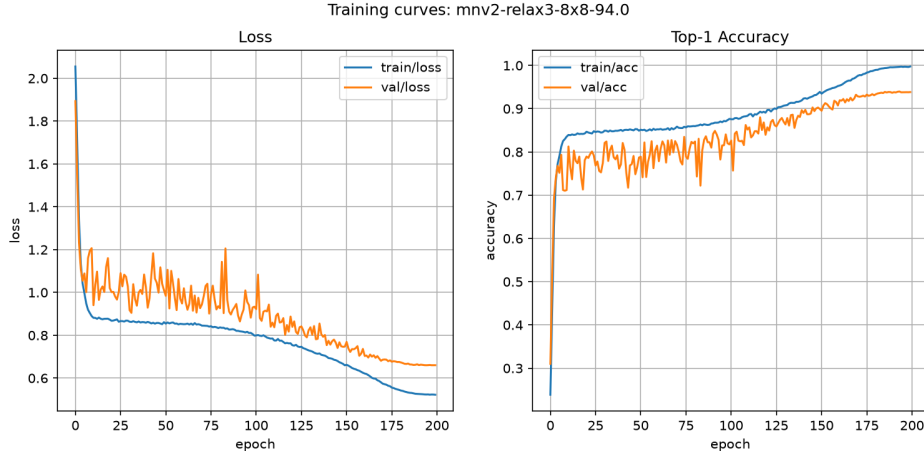


Figure 1: Training and validation loss and accuracy over 200 epochs.

Failure modes:

- **Over-downsampling:** The default stem/stride schedule destroys resolution on 32×32 inputs; this caps accuracy at 90.65% (`stride_relax=1`). Relaxing the early strides fixes it (+3.3 points).
- **Mild late overfitting:** A ~ 5 -point train-test gap opens in the final epochs (train $\approx 98.8\%$, test $\approx 94.0\%$), despite weight decay, dropout, label smoothing, and augmentation.
- **Class confusions:** Errors concentrate in the visually similar animal classes. The worst classes are cat (87.6%) and dog (90.1%); the top confusion is cat $\leftrightarrow$ dog.

2 Q2 — Model Compression Implementation

(a) Method and design choices

We implement **uniform affine post-training quantization** from scratch (no external quantization library). A value is quantized to an integer grid and immediately dequantized:

$$q = \text{clip}(\text{round}(x/s) + z, q_{\min}, q_{\max}), \quad \hat{x} = s(q - z),$$

with scale s and zero-point z .

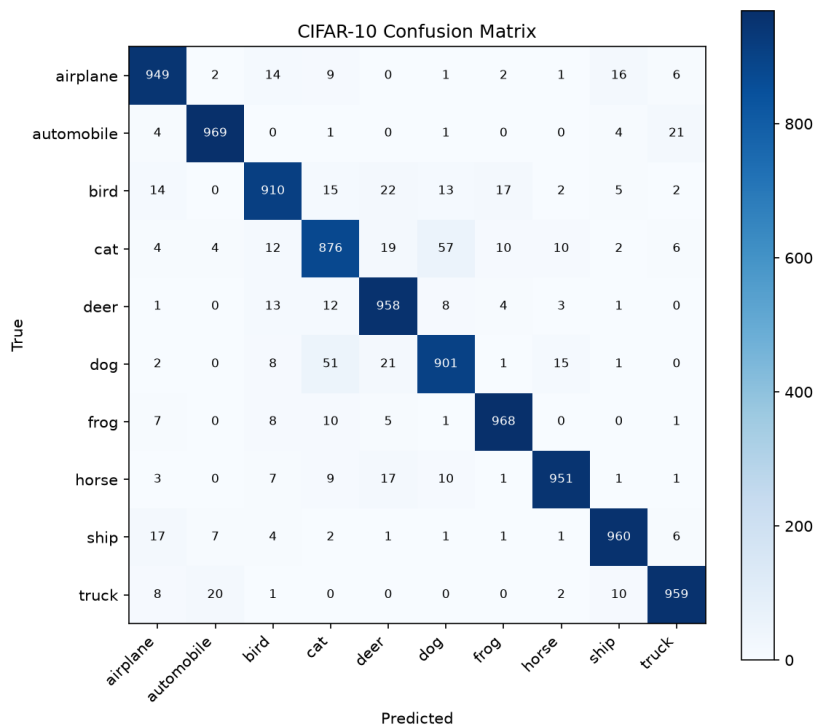


Figure 2: Confusion matrix on the 10k test set (overall accuracy 94.0%).

Design choices:

- **Fake Quantization:** We round each value, then convert it straight back to `float32`. This simulates low-bit storage while keeping the normal float forward path. This allows us to measure accuracy for any bit-width directly in PyTorch, with no integer kernels, and sweep many settings quickly.
- **Weights:** per-output-channel, *symmetric* ($z = 0$). Per-channel is essential because MobileNet-v2 depthwise convolutions have very different per-channel ranges.
- **Activations:** per-tensor, *affine* (asymmetric), with static calibration. Ranges are observed on a few hundred training images, then frozen.
- **Calibration:** MSE clipping — we search a clip fraction that minimizes the quantization error, which recovers accuracy at low bit-widths.

(b) Where it is applied (layers and exceptions)

Every `Conv2d` and `Linear` weight, and its input activation, is quantized, with these exceptions:

- The **first convolution** and the **final classifier** stay in `float32`.
- `BatchNorm2d` stays in `float32`.

(c) Storage overheads

The compressed size counts quantized weights at their bit-width, plus all metadata:

- **Weight scales:** one `float32` scale per output channel, per quantized layer ($\sum \text{out_channels} \times 32$ bits). Symmetric weights need no zero-points.
- **Activation qparams:** one `float32` scale and one `int32` zero-point per quantized tensor.
- **Float parameters:** `BatchNorm2d` parameters, biases, and the kept first/last layers are counted at 32 bits.

$$\text{Model size (MB)} = \frac{\sum(\text{weight bits}) + \text{metadata bits} + \text{float-param bits}}{8 \times 10^6}.$$

3 Q3 — Compression Results

We sweep weight bits $\in \{2, 3, 4, 6, 8\}$ and activation bits $\in \{4, 6, 8\}$ (Table 3). Figure 3 is the Weights & Biases parallel-coordinates chart of the sweep.

w-bits	a-bits	top-1 (%)	size (MB)	model CR	weight CR
8	8	93.83	2.449	3.65×	3.88×
6	8	93.65	1.901	4.71×	5.12×
4	8	90.85	1.354	6.61×	7.53×
4	6	89.28	1.354	6.61×	7.53×
3	8	40.73	1.080	8.28×	9.85×
2	8	10.00	0.807	11.09×	14.23×

4-bit activations collapse every row to ~ 10 – 17% .

Table 3: PTQ bit-width sweep

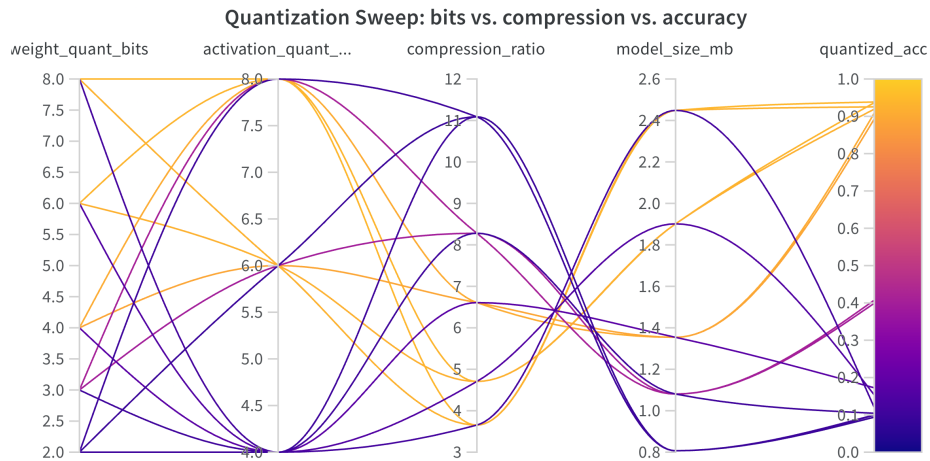


Figure 3: Wandb Parallel Coordinates plot of the quantization sweep.

Findings: Accuracy holds near the baseline down to 4-bit weights with ≥ 6 -bit activations. Two clear limits appear: 4-bit activations collapse accuracy (activations have a wider, spikier range than weights), and 2–3 bit weights collapse. These are the expected limits of PTQ without retraining.

4 Q4 — Compression Analysis

Selected Quant: 4-bit weights, 8-bit activations — the highest weight compression that still holds accuracy.

- (a) **Compression ratio of the weights: $7.53\times$.** We measure it as

$$\frac{P \times 32}{P \times w_bits + S \times 32},$$

where P is the number of quantized weight parameters and S is the number of per-channel scales (stored in `float32`; symmetric weights need no zero-points). At 4 bits this gives $7.53\times$, slightly below $32/4$ due to the scale overhead.

- (b) **Compression ratio of the activations: $4.0\times$.** We measure it over one forward pass as

$$\frac{\sum_i N_i \times 32}{\sum_i N_i \times \text{act_bits}} = \frac{32}{\text{act_bits}},$$

where N_i is the element count of the i -th quantized activation. At 8 bits this is $32/8 = 4.0\times$.

- (c) **Accuracy of the quantized model: 90.85% top-1.**
- (d) **Final approximated model size: 1.354 MB** (from 8.947 MB, a $6.61\times$ model compression ratio).

5 Q5 — Reproducibility & Repository

- (a) **Clean, modular codebase.** The code separates data ([src/data](#)), model, training, and evaluation ([src/models](#), [src/train.py](#), [src/eval.py](#)), and compression ([src/compression](#), [src/quantize.py](#)); tests live in [tests/](#).
- (b) **README, environment, dependency versions, seed.** The [README](#) lists the exact commands (below). The environment is recreated from `environment.yaml` with `conda env create`. Pinned versions: Python 3.12, torch 2.14.0+cu132, torchvision 0.29.0+cu132, lightning 2.6.5, torchmetrics 1.9.0, hydra-core 1.3.6, wandb 0.29.0. The seed is 42, set in `configs/experiment/mobilenetv2_cifar10.yaml`; Lightning seeds Python, NumPy, and PyTorch. Exact commands to reproduce the results:

```
conda env create -f environment.yaml && conda activate cs6886
python src/train.py experiment=mobilenetv2_cifar10           # Q1 baseline
python src/quantize.py compression.weight_bits=4 \
                    compression.activation_bits=8           # Q2 quantize (w4a8)
bash scripts/sweep.sh                                       # Q3 bit-width sweep
```

- (c) **Repository link:** https://github.com/starkAhmed43/CS6886_A2